Theoretische Informatik

Laurin Seeholzer

October 29, 2026

Contents

1 Introduction **3**
- 1.1 Turing Machines . . . 3
- 1.2 P vs. NP . . . 4
- 1.3 Polynomial Time Reductions . . . 4

2 Finite Automata **5**
- 2.1 Symbol, Alphabets and Words . . . 5
- 2.2 Formal Languages . . . 6
- 2.3 Algorithmic Problems . . . 6
- 2.4 Finite Automata . . . 6
- 2.5 Non-regularity . . . 8
- 2.6 Nondeterminism . . . 9
- 2.7 Power-set-construction . . . 10

3 Turing Machines **12**
- 3.1 Basic Concept . . . 12
- 3.2 Formal Definition . . . 12
- 3.3 Configurations . . . 13
- 3.4 Computation and Steps . . . 13
- 3.5 Multi-Tape Turing Machines . . . 14
- 3.6 Nondeterministic Turing Machines . . . 16
- 3.7 Recognizability and Decidability . . . 17
- 3.8 Universal Turing Machine and Universal Language . . . 18
- 3.9 The Diagonal Language (L_{diag}) and Its Complement (L^c_{diag}) . . . 18

4 Reductions and Undecidability **20**
- 4.1 The Concept of Reductions . . . 20
- 4.2 General Reductions (R-Reductions) . . . 20
- 4.3 Many-One Reductions (EE-Reductions) . . . 20
- 4.4 Canonical Undecidable Problems and Classic Reductions . . . 22
- 4.5 The Emptiness Problem and Non-Recognizability . . . 22
- 4.6 Rice's Theorem . . . 23

1 Introduction

Leibnitz proposed that some time in the future, mathematics will be developed enough to model and solve every possible problem. Then came Russel

Definition 1.1. Russels Paradox: Let S be the set of sets that do not contain themselves.

$$S = X | X \notin X$$

Russels Paradox poses the following Question: Does S contain intself. The answer is paradoxical

$$S \notin S \implies S \in S$$
$$S \in S \implies S \notin S$$

Remark 1.2. Hilbert style deduction allows you to solve Russels paradox by first stating what the axioms and interference rules of the System are.

Theorem 1.3. Goedels incompletness: There are mathematical statements that are true but can not possibly be proven to be so.

1.1 Turing Machines

Definition 1.4. Turing-Machine (TM): A theoretical machine consisting of an infinity strand of paper, a perfect pen and a set of rules. Every step includes reading current position, modifying it, moving left/right.

Remark 1.5. It can be shown that every algorithm can be modeld as a Turing-Machine and vice-versa.

Theorem 1.6. Turing-Theorem: There are problems that can not be automatically solved using an algorithm/TM.

Proof. Let $w_{i,1}, w_{i,2}, w_{i,3} \dots$ be the output of Turing Machine i for inputs $1, 2, 3 \dots$. We define a new Turing Machine M_x such that $w_{x,j} \neq w_{j,j}$ hence the output of M_x differs from the output of every other Turing machine for at least one input. Observe that this is a valid Turing Machine but it can not possibly be contained in our current list of Turing Machines. Hence, we reach a contradiction: if M_x were in our enumeration at some index x, its output on input x would be $w_{x,x} \neq w_{x,x}$, which is logically impossible. Therefore, our assumption that we can construct such a machine M_x must be false. This implies there is no computable general algorithm capable of determining the output $w_{j,j}$ for all

Turing machines j, proving that there exist uncomputable functions and that the Halting Problem is undecidable. □

Example 1.7. Halting-problem: Does algorithm A terminate?
This problem seems easy at first but we can design programs that for example only terminate once they find an even number n such that there are no primes p, q with $n = p + q$. Solving this (finding such n) would solve the Goldbach-Conjecture which we currently do not know to be true/false.

1.2 P vs. NP

Definition 1.8. P vs. NP: The p/np problem poses the question if all in polynomial time verifiable problems (NP) have a polynomial time solution (P).

Definition 1.9. A proiblem A in NP is called **NP-Complete** if the existence of an efficient solution for A implies the existance of an efficient solution of all NP problems.

1.3 Polynomial Time Reductions

Definition 1.10. A clique in a graph is a set of vertices sucht that they are all connected to each other.

Definition 1.11. An independent-Set in a Graph is a set of vertices, such that none of them are connected to each other.

Example 1.12. Assume we know "does G have a clique of size k" is NP-Coplete. How can we show that then "Does G have a stable set of size k" is also NP-Complete.

Proof. Observe that inverting a graph (Adding edges where there are none and removing edges where there are) creates a graph G' such that every clique in G is a stable Set in G' and viceversa. This lets us calculate stable sets from cliques and vice verse hence if one problem is NP-Complete both have to be. □

2 Finite Automata

2.1 Symbol, Alphabets and Words

Definition 2.1. Alphabet: finite, non-empty set.

Example 2.2. Examples of Alphabets:

- Boolean alphabet $0, 1$
- latin alphabet $a, b, \ldots, z$
- logical alphabet $0, a, x, (,), \ldots$

Definition 2.3. Word: A word over an alphabet A is a finite sequence of symbols of A.

Definition 2.4. The **lenght** of a word it the number of symbols it uses. There is always an empty word of length zero.

Remark 2.5. Words can be concatenated or rewersed to form new words.

$$kon(x, y) = xy$$

$$w = a_1a_2a_3 \implies w^R = a_3a_2a_1$$

Definition 2.6. The i-th iteration of a word $x = ab$ is $x^i = abab\ldots ab$ the word concatenated with itself i times.

Definition 2.7. A word v is a partial-, prefix- or suffix-word of a word w if v is contained in w, w begins with v or ends with v.

Remark 2.8. For a word x and symbol a the expression $|x|_a$ describes the numnber of apperances of a in x.

Theorem 2.9. For an alphabet with an order imposed on its symbols ($a_1 \leq a_2 \leq ...$) we can define the harmonic order of its words as a lexicographic order where

$$u \leq v \iff (|u| \leq |v|) \vee (|u| = |v| \wedge u = xs_iu', v = xs_jv'.s_i \leq s_j)$$

2.2 Formal Languages

Definition 2.10. A subset $L \subset \Sigma^*$ is called Language over the alphabet Σ

Definition 2.11. The **complement** L^C of a Language L is definied as $\Sigma^* \setminus L$

Definition 2.12. The **concatenation** of two languages L_1, L_2 is all the set of all $w_1 w_2$ where $w_1 \in L_1, w_2 \in L_2$

Remark 2.13. We can concatenate a Language with itself (possibly multiple times or zero)

$$L^0 = \{\lambda\}, L^1 = L, L^2 = LL \dots L^* = \bigcup_{i=0}^{\infty} L^i, L^+ = LL^*$$

Lemma 2.14. Let L_1, L_2, L_3 be languages over an alphabet Σ then

$$L_1 L_2 \cup L_1 L_3 = L_1 (L_2 \cup L_3)$$

$$L_1 (L_2 \cap L_3) \subseteq L_1 L_2 \cap L_1 L_3$$

2.3 Algorithmic Problems

Remark 2.15. An algorithm describes a function $\Sigma^*_{in} \to \Sigma^*_{out}$

Definition 2.16. Let Σ be an alphabet and $L \subseteq \Sigma^*$. The decision problem (Σ, L) is defined (for arb $x \in Sigma^*$) as

$$x \in L \quad \text{or} \quad x \notin L$$

Remark 2.17. If an algorithm A correctly decides for any x then we say

$$A \quad \text{detects the Language} \quad L$$

2.4 Finite Automata

Remark 2.18. A finite automata (finite state machine) is a set of states together with state transitions on inputs with fixed boolean outputs on each state.

Definition 2.19. Finite Automata: is a Quadroupel $M = (Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states
- Σ is an alphabet (Input-alphabet)
- δ is a function $Q \times \Sigma \to Q$
- $q_0 \in Q$ is the starting-state
- $F \subseteq Q$ is the set of accepting (true) states

Definition 2.20. Configuration: of an Automata is an Elment of $Q \times \Sigma^*$

- **Starting-Configuraiton:** $C_0 = (q_0, x) | x \in \Sigma^*$
- **End-Configruation:** $C_n = (q_0, \lambda)$ where its accepting if $C_n \in F \times \{\lambda\}$

Remark 2.21. A calculation can be seen as a finite sequence of configurations $C_1, C_2 \dots$ such that for every $C_i \to_{s_M} C_{i+1}$ with steps s_M in our machine M

$$(q_0, 01001) \vdash_M (q_x, 1001) \vdash_M (q_y, 001) \dots$$

where a step is a relation on configurations

$$\vdash_M \subseteq (Q \times \Sigma^*) \times (Q \times \Sigma^*)$$

Definition 2.22. The **transitive hull of transitions** describes the calculation over multiple steps:

$$(q, w) \vdash_M^* (p, u) \iff (q = p \wedge w = u) \vee \exists k \in \mathbb{N}^+ \text{ s.t.}$$

$$w = a_1 \dots a_k u \quad \text{with} \quad a_i \in \Sigma \quad \text{and}$$

$$\exists r_1, \dots, r_{k-1} \in Q \text{ s.t. } (q, w) \vdash_M (r_1, a_2 \dots a_k u) \vdash_M (r_2, a_3 \dots a_k u) \cdots \vdash_M (p, u)$$

Definition 2.23. The **extended transition function** evaluates the end state of a word over multiple transitions:

$$\tilde{\delta} : Q \times \Sigma^* \to Q$$

$$\tilde{\delta}(q, \lambda) = q \quad \forall q \in Q$$

$$\tilde{\delta}(q, wa) = \delta(\tilde{\delta}(q, w), a) \quad \forall a \in \Sigma, w \in \Sigma^*, q \in Q$$

Remark 2.24. States can be grouped into equivalence classes containing all elements with the same end state. This is useful for proofs regarding the language of finite automata, which are typically done through these classes and induction.

Definition 2.25. Product Automata: Used to determine the intersection, union or exclusion ($\odot$) of two finite automata (their languages), built with the state space $Q = Q_1 \times Q_2$.

$$M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1), \quad M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$$

$$\implies M = (Q_1 \times Q_2, \delta((x, y), a) = (\delta_1(x, a), \delta_2(y, a)), (q_01, q_02), F_1 \odot F_2)$$

Remark 2.26. The correctness of this product automata can be proven by induction over the word length.

2.5 Non-regularity

Definition 2.27. A language L is called **regular** if there exists a finite automata that accepts/detects the language L and **non-reular** else.

Lemma 2.28. Let $A = (Q, \Sigma, \delta, q_0, F)$ be a finite automata and let $x, y \in \Sigma^*$ such that

$$\tilde{\delta}(q_0, x) = \tilde{\delta}(q_0, y)$$

then there exists $z \in \Sigma^*$ such that

$$Kl[xz] = Kl[yz]$$

Remark 2.29. This illustrates that finite machines are finite hence can not distinguish beetween xz and yz once they are in a certain state q_r, they do not know how they got there.

Proof. **Proof of Non-Regularity:** Let $L = \{0^n 1^n | n \in \mathbb{N}\}$ and assume $L \in L_{EA}$ (L is regular) hence there exists a finite automata A such that $L(A) = L$. Let $m = |Q|$ and observe the words $x \in \{0^n | n \leq m + 1\}$. Because this are more words than A has states there exists $i \neq j \in [0, m + 1] \cap \mathbb{N}$ such that

$$\tilde{\delta}(q_0, 0^i) = \tilde{\delta}(q_0, 0^j)$$

Hence by the above lemma the words $0^i1^i, 0^j1^i$ will both be accepted which contradicts the assumption that A detects L because $0^j1^i \notin L$. □

Lemma 2.30. Pumping-Lemma: Let a language $L \in \Sigma^*$ be regular, then there exists a constant $n_0 \in \mathbb{N}$ such that every word $w \in \Sigma^*$ with $|w| \geq n_0$ can be taken apart into x, y, z where

- $|yx| \leq n_0$
- $|x| \geq 1$
- $\{yx^kz|k \in \mathbb{N}\} \subseteq L \vee \{yx^kz|k \in \mathbb{N}\} \cap L = \emptyset$

Example 2.31.

$$L = 0^n1^n|n \in \mathbb{N}$$

Proof using Pumping Lemma: Assume L regular hence there exists constant n_0 with the properties described by the pumping lemma.

$$w = 0^{n_0}1^{n_0}$$

And there exists $yxz = w$ that fulfill the properties of the pumping lemma. Hence

$$|yx| \leq n_0 \implies y = 0^e, x = 0^m \quad e, m \in \mathbb{N}$$

$$|x| \geq 1 \implies m \geq 1$$

$$|yxz| = n_0 \implies \{0^{n_0-m+km}1_0^n|k \in \mathbb{N}\} \subseteq L$$

Observe that this is a contradiction as for $k = 0$ the last condition does not hold.

$$yx^0z = yz = 0^{n_0-m}1^{n_0} \notin L$$

2.6 Nondeterminism

Important 2.32. Nondeterminism has nothing to do with Non Deterministic (random) behavior.

Definition 2.33. Nondeterministic finite automata are finite automatas where the transition-function δ is not well definied hence

$$\hat{\delta}(q_k, w) = \{q_i, q_j...\}$$

and a

$$w \in L \iff \exists q_l \in F \cap \{q_i, q_j...\}$$

Hence a word is accepted if there is at least one valid path that leads to an accepting endstate

2.7 Power-set-construction

Theorem 2.34. Power-Set Construction (Subset Construction): For every nondeterministic finite automaton $M = (Q_N, \Sigma, \delta_N, q_0, F_N)$, there exists an equivalent deterministic finite automaton $M_D = (Q_D, \Sigma, \delta_D, q_{0,D}, F_D)$ such that

$$L(M_D) = L(M)$$

Definition 2.35. Given an NFA $M = (Q_N, \Sigma, \delta_N, q_0, F_N)$, the equivalent DFA $M_D = (Q_D, \Sigma, \delta_D, q_{0,D}, F_D)$ is constructed as follows:

- **States:** $Q_D = \mathcal{P}(Q_N)$ (the power set of Q_N, i.e., every state in M_D is a subset $S \subseteq Q_N$)
- **Alphabet:** Σ (identical to the NFA's alphabet)
- **Start State:** $q_{0,D} = \{q_0\}$
- **Transition Function:** For every $S \in Q_D$ and $a \in \Sigma$:

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$$

- **Accepting States:** A state $S \in Q_D$ is accepting if it contains at least one accepting state of M:

$$F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$$

Remark 2.36. The intuition behind the construction is that the DFA tracks all possible states the NFA could currently be in simultaneously. While the power set has $|Q_D| = 2^{|Q_N|}$ states, in practice only the states reachable from $\{q_0\}$ need to be constructed. By induction on word length $|w|$, one can show that

$$\tilde{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w)$$

which immediately implies $w \in L(M_D) \iff w \in L(M)$.

Theorem 2.37. State Explosion / Exponential Lower Bound: For every $k \in \mathbb{N}^+$, let L_k be the language over $\Sigma = \{0, 1\}$ defined by:

$$L_k = \{w \in \{0,1\}^* \mid \text{the } k\text{-th symbol from the end of } w \text{ is } 1\}$$

Then:

1. There exists an NFA accepting L_k with only $k + 1$ states.
2. Every DFA accepting L_k requires at least 2^k states.

Example 2.38. NFA for L_k: We can construct an NFA $M = (Q, \{0,1\}, \delta, q_0, \{q_k\})$ with $Q = \{q_0, q_1, \ldots, q_k\}$:

- $\delta(q_0, 0) = \{q_0\}$
- $\delta(q_0, 1) = \{q_0, q_1\}$ (nondeterministically guess that this 1 is the k-th symbol from the end)
- $\delta(q_i, 0) = \delta(q_i, 1) = \{q_{i+1}\}$ for $1 \leq i \leq k-1$

This automaton has exactly $k+1$ states and accepts L_k.

Proof. **Proof ntuition for the 2^k DFA Lower Bound:** Unlike an NFA, a deterministic automaton cannot guess when it is k symbols away from the end. Since the input could stop at any time, a DFA must maintain the exact history of the last k symbols seen. There are 2^k possible words of length k, so it must distinguish all of them. □

3 Turing Machines

3.1 Basic Concept

Remark 3.1. A Turing Machine (TM) is a fundamental generalization of finite automata:

- **Finite Control:** Like a finite automaton, a TM possesses a finite set of internal states Q.
- **Unbounded Memory (Tape):** Instead of a one-way read-only input, a TM has an infinite (or semi-infinite) tape divided into discrete cells, serving as both input medium and unbounded memory.
- **Read/Write Head:** A movable head that can read the symbol in the current cell, overwrite it with a new symbol, and move either left (L) or right (R) along the tape.

3.2 Formal Definition

Definition 3.2. Turing Machine: A deterministic Turing Machine is a 7-tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

where:

- Q is a finite, non-empty set of states.
- Σ is a finite input alphabet.
- Γ is a finite tape alphabet such that $\Sigma \subset \Gamma$. It contains:
  - the **blank symbol** $\sqcup \in \Gamma \setminus \Sigma$ (representing empty cells)
  - the **left-end marker** $¢ \in \Gamma \setminus \Sigma$ (marking the left boundary of the tape)
- δ is the transition function:

  $$\delta : (Q \setminus \{q_{accept}, q_{reject}\}) \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\}$$

  with the convention that upon scanning the left-end marker ¢, the machine cannot overwrite it and must move right:

  $$\delta(q, ¢) = (p, ¢, \mathrm{R}) \quad \text{for some } p \in Q$$

- $q_0 \in Q$ is the starting state.
- $q_{accept} \in Q$ is the accepting halt state.
- $q_{reject} \in Q$ is the rejecting halt state ($q_{accept} \neq q_{reject}$).

3.3 Configurations

Definition 3.3. Configuration: A configuration of a Turing Machine captures its current global state, consisting of the tape contents up to the last relevant cell, the current state q, and the head position. Formally, the set of all configurations is:

$$\text{Konf}(M) = \{¢\} \cdot \Gamma^* \cdot Q \cdot \Gamma^+ \ \cup \ Q \cdot \{¢\} \cdot \Gamma^+$$

In this word representation, the state symbol $q \in Q$ is written immediately to the left of the tape cell currently scanned by the read/write head:

- If the head scans the left-end marker ¢:

$$C = q¢u \in Q \cdot \{¢\} \cdot \Gamma^+$$

- If the head scans cell $i \geq 1$ with tape prefix $¢u$:

$$C = ¢uqv \in \{¢\} \cdot \Gamma^* \cdot Q \cdot \Gamma^+$$

Definition 3.4. Start Configuration: For an input word $x \in \Sigma^*$, the start configuration is:

$$C_0(x) = \begin{cases} q_0¢x⊔ & \text{if } x \neq \lambda \\ q_0¢⊔ & \text{if } x = \lambda \end{cases}$$

The machine starts in state q_0 scanning the left-end marker ¢, followed by the input x and blank symbols ⊔.

3.4 Computation and Steps

Definition 3.5. Computation Step: A single step is a transition relation $\vdash_M$ between configurations:

- **Move Right (R):** If $\delta(q, a) = (p, b, \text{R})$, then for all $u \in \Gamma^*, v \in \Gamma^+$:

$$uqav \vdash_M ubpv$$

  (if $v = \lambda$, the cell is implicitly filled with ⊔).

- **Move Left (L):** If $\delta(q, a) = (p, b, \text{L})$, then for all $u \in \Gamma^*, c \in \Gamma, v \in \Gamma^+$:

$$ucqav \vdash_M upcbv$$

We denote by $\vdash_M^*$ the reflexive and transitive closure of $\vdash_M$ (computation over multiple steps).

Definition 3.6. Computation (Calculation): A computation of M on input x is a sequence of configurations $C_0, C_1, C_2, \ldots$ where $C_0 = C_0(x)$ and $C_i \vdash_M C_{i+1}$ for all $i \geq 0$.

- **Halting:** The computation terminates if the machine reaches a halting state (q_{accept} or q_{reject}), since δ is not defined for these states.
- **Infinite Loop:** If no halting state is ever reached, the computation runs forever (M does not terminate on x).

3.5 Multi-Tape Turing Machines

Definition 3.7. Multi-Tape Turing Machine (k-Tape TM): A k-tape Turing Machine ($k \geq 1$) consists of a finite control and k independent, semi-infinite tapes, each equipped with its own independently moving read/write head:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

where the transition function is defined as:

$$\delta : (Q \setminus \{q_{accept}, q_{reject}\}) \times \Gamma^k \to Q \times \Gamma^k \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}^k$$

In a single computation step, the machine reads the k-tuple of symbols currently under the k heads $(a_1, \ldots, a_k) \in \Gamma^k$, transitions to a new state $p \in Q$, writes new symbols $(b_1, \ldots, b_k) \in \Gamma^k$, and moves each head independently according to $(D_1, \ldots, D_k) \in \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}^k$ (where S denotes staying stationary).

Remark 3.8. Start Configuration: On input $x \in \Sigma^*$, Tape 1 contains the input word preceded by the left-end marker $¢x \sqcup \ldots$, while all other $k-1$ work tapes are initially empty ($¢ \sqcup \ldots$). All k heads initially scan the left-end marker $¢$.

Remark 3.9. Why Use Multi-Tape Machines?

- **Natural Algorithm Design:** A multi-tape TM provides a much more intuitive programming paradigm than a single-tape TM. Dedicated tapes can be designated for specific roles (e.g., an input tape, one or more scratch/work tapes, and an output tape).
- **Modularity:** Multiple tapes allow implementing modular subroutines (e.g., copying strings, arithmetic, or counting) without continually traversing and corrupting the primary input.

Theorem 3.10. Equivalence of Single-Tape and Multi-Tape TMs: Multi-tape Turing machines have exactly the same computational power as standard single-tape

Turing machines. A language L is Turing-recognizable (resp. decidable) by a k-tape TM if and only if it is Turing-recognizable (resp. decidable) by a single-tape TM:

$$\mathcal{L}(\text{Multi-Tape TM}) = \mathcal{L}(\text{Single-Tape TM})$$

Remark 3.11. Simulation via Tracks (Spurtechnik): A single-tape TM S can simulate a k-tape TM M by dividing its single tape into $2k$ tracks using an expanded tape alphabet $\Gamma_S \cong (\Gamma \times \{0,1\})^k$:

- For each tape $i \in \{1, \dots, k\}$, one track stores the contents of tape i, and a second track stores a head marker flag (a bit 1 or a marked symbol $\hat{a}$) indicating the current position of head i.
- To simulate one step of M, machine S sweeps across the tape from the leftmost head marker to the rightmost head marker, records all k scanned symbols in its finite control, applies δ, and sweeps back to write the replacement symbols and shift the head markers.

Definition 3.12. Church-Turing Thesis (Church'sche These):

Every algorithmically computable function or intuitive effective procedure can be carried out by a standard Turing Machine.

Remark 3.13. Significance and Robustness:

- Augmenting a Turing machine with multiple tapes, multi-dimensional tapes, random-access memory (RAMs), or non-determinism does not expand the class of decidable languages. This demonstrates the exceptional robustness of the Turing machine model.
- Independent models of computability (λ-calculus, μ-recursive functions, RAMs, and modern programming languages) define the exact same class of computable functions.
- **Extended Church-Turing Thesis:** Any physically realistic computational model can be simulated by a deterministic single-tape Turing machine with at most a polynomial slowdown. Because simulating a k-tape TM on a 1-tape TM incurs only a quadratic slowdown, the multi-tape TM fully complies with the Extended Church-Turing Thesis.

Definition 3.14. Time Complexity: Let $T : \mathbb{N} \to \mathbb{N}$. A Turing Machine M runs in time $T(n)$ if for every input word $x \in \Sigma^*$ of length $n = |x|$, the computation of M on x halts within at most $T(n)$ steps.

Theorem 3.15. Quadratic Simulation Theorem (Hartmanis & Stearns): Let M be a k-tape Turing machine that decides a language L in time $T(n)$. Then there exists a single-tape Turing machine S that decides L in time $\mathcal{O}(T(n)^2)$:

$$\text{TIME}_{k\text{-tape}}(T(n)) \subseteq \text{TIME}_{1\text{-tape}}(T(n)^2)$$

Remark 3.16. Derivation of Quadratic Overhead:

- In $T(n)$ steps, each head of M can travel at most distance $T(n)$ from the left-end marker. Hence, all k head markers on the single tape of S are contained within a span of at most $k \cdot T(n) = \mathcal{O}(T(n))$ cells.
- For every single step of M, machine S sweeps over this span to read the symbols under the k head markers and sweeps back to update them, requiring $\mathcal{O}(T(n))$ steps per simulated step.
- Total simulation time for $T(n)$ steps is therefore:

$$T(n) \cdot \mathcal{O}(T(n)) = \mathcal{O}(T(n)^2)$$

Example 3.17. Separation: Palindrome Recognition (L_{pal}): Consider the palindrome language $L_{pal} = \{w \in \{0,1\}^* \mid w = w^R\}$:

- **2-Tape TM:** Copies w to Tape 2 in n steps, moves head 2 back to the left in n steps, and compares Tape 1 and Tape 2 in n steps. Total time: $\mathcal{O}(n)$ (linear time).
- **1-Tape TM:** Must shuttle back and forth between the opposing ends of w to compare characters, which takes $\Omega(n^2)$ steps. (Formally proven via crossing sequences).

This proves that a 2-tape TM can achieve a quadratic speedup over a 1-tape TM, and the $\mathcal{O}(T(n)^2)$ simulation bound is asymptotically tight.

Remark 3.18. Invariance of Class P: Because the overhead of simulating multi-tape TMs on single-tape TMs is polynomial ($\mathcal{O}(T(n)^2)$), the fundamental complexity class $\mathbf{P}$ (problems solvable in polynomial time) is invariant under the number of tapes:

$$\mathbf{P}_{1\text{-tape}} = \mathbf{P}_{k\text{-tape}}$$

3.6 Nondeterministic Turing Machines

Definition 3.19. Nondeterministic Turing Machine (NTM): An NTM is defined similarly to a deterministic TM, but its transition function allows multiple alternative transitions for any given state and tape symbol:

$$\delta : (Q \setminus \{q_{accept}, q_{reject}\}) \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\text{L}, \text{R}\})$$

At each step, the machine may choose any transition from $\delta(q, a)$. If $\delta(q, a) = \emptyset$, that

computation branch halts without accepting.

Definition 3.20. Computation Tree and Acceptance: The set of all possible computations on an input x forms a **computation tree** rooted at the start configuration $C_0(x)$. An NTM M **accepts** x if **at least one** computation path in the tree halts in q_{accept}:

$$x \in L(M) \iff \exists \text{ path s.t. } C_0(x) \vdash_M^* uq_{accept}v \quad (\text{for some } u, v \in \Gamma^*)$$

M rejects x if all computation branches halt in q_{reject} (or fail to reach q_{accept}).

Theorem 3.21. Equivalence of DTMs and NTMs: For every nondeterministic Turing machine N, there exists an equivalent deterministic Turing machine D such that:

$$L(D) = L(N)$$

Hence, nondeterminism does not increase the computational power of Turing machines (the class of recognizable languages remains identical).

Remark 3.22. BFS Simulation and Time Complexity:

- **Breadth-First Search (BFS):** The deterministic simulator D systematically explores the computation tree of N level by level. A depth-first search would not suffice, as it could get trapped in an infinite non-accepting branch.
- **Exponential Overhead:** If the shortest accepting path has length $T(n)$ and the maximum branching factor is b, the tree contains up to $b^{T(n)}$ configurations. The deterministic simulation runs in **exponential time** $\mathcal{O}(b^{T(n)})$, which motivates the $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ question.

3.7 Recognizability and Decidability

Definition 3.23. Accepted Language: A word $x \in \Sigma^*$ is accepted by M if the computation of M on x halts in q_{accept}:

$$L(M) = \{x \in \Sigma^* \mid C_0(x) \vdash_M^* uq_{accept}v \text{ for some } u, v \in \Gamma^*\}$$

Definition 3.24. Decidable (Recursive) Language: A language $L \subseteq \Sigma^*$ is called **decidable** (or **recursive**) if there exists a Turing Machine M that **halts on every input** $x \in \Sigma^*$ such that:

$$x \in L \iff M \text{ halts in } q_{accept}$$

$$x \notin L \iff M \text{ halts in } q_{reject}$$

A Turing machine with this property is called a **decider**.

Definition 3.25. Turing-Recognizable (Recursively Enumerable) Language: A language $L \subseteq \Sigma^*$ is called **Turing-recognizable** (or **recursively enumerable** / semi-decidable) if there exists a Turing Machine M such that:

$$L(M) = L$$

For words $x \notin L$, M is allowed to halt in q_{reject} or loop indefinitely.

Remark 3.26. Every decidable language is Turing-recognizable, but the converse does not hold. For example, the Halting Problem $H = \{\langle M, w\rangle \mid M \text{ halts on } w\}$ is Turing-recognizable but undecidable.

Theorem 3.27. Decidability via Complements: A language $L \subseteq \Sigma^*$ is **decidable** if and only if both L and its complement $L^c = \Sigma^* \setminus L$ are **Turing-recognizable**:

$$L \in \mathcal{L}_{\text{rec}} \iff L \in \mathcal{L}_{\text{RE}} \wedge L^c \in \mathcal{L}_{\text{RE}}$$

3.8 Universal Turing Machine and Universal Language

Definition 3.28. Universal Language (L_{univ}): Let $\langle M\rangle \in \{0,1\}^*$ denote a binary encoding of a Turing machine M. The **Universal Language** L_{univ} (also denoted L_U or A_{TM}) consists of all pairs of machine encodings and input words accepted by that machine:

$$L_{\text{univ}} = \{\langle M\rangle \# w \mid w \in L(M)\}$$

3.9 The Diagonal Language (L_{diag}) and Its Complement (L^c_{diag})

Definition 3.29. Canonical TM Enumeration: Let $w_1, w_2, w_3, \ldots$ be the canonical lexicographic ordering of all words in $\{0,1\}^*$. Every word w_i can be uniquely interpreted as an encoding of a Turing machine $M_i = M_{w_i}$ (if w_i is not a syntactically valid encoding, M_i is defined as a dummy TM with $L(M_i) = \emptyset$).

Definition 3.30. Diagonal Language (L_{diag}): The diagonal language contains all words w_i whose corresponding Turing machine M_i does **not** accept its own encoding

w_i:

$$L_{\text{diag}} = \{w_i \in \{0,1\}^* \mid w_i \notin L(M_i)\}$$

Theorem 3.31. Non-Recognizability of L_{diag} (Diagonalization): The diagonal language is **not Turing-recognizable**:

$$L_{\text{diag}} \notin \mathcal{L}_{\text{RE}}$$

Proof. Assume for contradiction that L_{diag} is Turing-recognizable. Then there exists a Turing machine M_d such that $L(M_d) = L_{\text{diag}}$. Since every TM appears in the enumeration, $M_d = M_{w_d}$ for some word w_d. Now consider the input w_d:

$$w_d \in L_{\text{diag}} \iff w_d \notin L(M_{w_d}) \iff w_d \notin L_{\text{diag}}$$

This is an immediate contradiction ($X \iff \neg X$). Hence, no such machine M_d can exist. □

Definition 3.32. Complement of L_{diag} (L^c_{diag}, Self-Acceptance): The complement of L_{diag} consists of all words w_i whose corresponding Turing machine M_i **accepts** w_i:

$$L^c_{\text{diag}} = \{0,1\}^* \setminus L_{\text{diag}} = \{w_i \in \{0,1\}^* \mid w_i \in L(M_i)\}$$

Theorem 3.33. Properties of L^c_{diag}:

- **Turing-Recognizable ($L^c_{\text{diag}} \in \mathcal{L}_{\text{RE}}$):** On input w_i, a machine can decode w_i, construct M_i, and simulate M_i on input w_i using a universal Turing machine. If M_i accepts w_i, the simulation halts and accepts.
- **Undecidable ($L^c_{\text{diag}} \notin \mathcal{L}_{\text{rec}}$):** If L^c_{diag} were decidable, its complement $(L^c_{\text{diag}})^c = L_{\text{diag}}$ would also be recognizable by the complement theorem, contradicting $L_{\text{diag}} \notin \mathcal{L}_{\text{RE}}$.

4 Reductions and Undecidability

4.1 The Concept of Reductions

Remark 4.1. Core Idea: Reductions are the primary tool in computability and complexity theory to compare the relative difficulty of algorithmic problems. If a problem A can be solved using an algorithm (subroutine) for a problem B, we say that A **reduces to** B ($A \leq B$). This means:

$$\text{Problem } B \text{ is at least as hard as problem } A.$$

Important 4.2. The Two Ways to Use a Reduction ($A \leq B$):

- **Positive Direction (Algorithm Design / Decidability):** If problem B is known to be decidable, then problem A is also decidable:

$$A \leq B \ \wedge \ B \in \mathcal{L}_{\text{rec}} \implies A \in \mathcal{L}_{\text{rec}}$$

- **Negative Direction (Impossibility Proofs / Undecidability):** By contraposition, if problem A is known to be undecidable, then problem B must also be undecidable:

$$A \leq B \ \wedge \ A \notin \mathcal{L}_{\text{rec}} \implies B \notin \mathcal{L}_{\text{rec}}$$

  This negative direction is the main method used to establish that newly encountered computational problems are undecidable.

4.2 General Reductions (R-Reductions)

Definition 4.3. R-Reduction (Turing / Oracle Reduction): Let $L_1 \subseteq \Sigma_1^*$ and $L_2 \subseteq \Sigma_2^*$. We say that L_1 is **R-reducible** to L_2 (denoted $L_1 \leq_R L_2$) if there exists an algorithm (or Turing machine with an oracle for L_2) that decides membership in L_1, using queries to an imaginary subroutine for L_2 as a black box.

Theorem 4.4. Properties of R-Reductions:

- **Reflexivity and Transitivity:** $L \leq_R L$, and if $L_1 \leq_R L_2$ and $L_2 \leq_R L_3$, then $L_1 \leq_R L_3$.
- **Decidability Transfer:** If $L_1 \leq_R L_2$ and $L_2 \in \mathcal{L}_{\text{rec}}$, then $L_1 \in \mathcal{L}_{\text{rec}}$.
- **Complement Invariance:** $L \leq_R L^c$ always holds (simply invert the yes/no answer of the subroutine).

4.3 Many-One Reductions (EE-Reductions)

Definition 4.5. EE-Reduction (Eingabe-Eingabe / Many-One Reduction): A language $L_1 \subseteq \Sigma_1^*$ is called **EE-reducible** to $L_2 \subseteq \Sigma_2^*$ (written $L_1 \leq_{EE} L_2$ or $L_1 \leq_m L_2$) if there exists a **computable total function**

$$f : \Sigma_1^* \to \Sigma_2^*$$

such that for every word $w \in \Sigma_1^*$:

$$w \in L_1 \iff f(w) \in L_2$$

The function f is computed by a halting deterministic Turing machine M_f (a transducer).

Remark 4.6. Intuition behind the Name: It is called an *Eingabe-Eingabe-Reduktion* (input-to-input reduction) because the reduction function f directly maps an input w of problem L_1 into a corresponding input $f(w)$ of problem L_2, without any post-processing of the decision result:

$$\text{Decider for } L_1(w) = \text{Decider for } L_2(f(w))$$

Every EE-reduction is a special case of an R-reduction: $L_1 \leq_{EE} L_2 \implies L_1 \leq_R L_2$.

Theorem 4.7. Conservation Theorems for EE-Reductions: Let $L_1 \leq_{EE} L_2$ with reduction function f.

1. **Decidability:**
   - If $L_2 \in \mathcal{L}_{\text{rec}}$, then $L_1 \in \mathcal{L}_{\text{rec}}$.
   - If $L_1 \notin \mathcal{L}_{\text{rec}}$, then $L_2 \notin \mathcal{L}_{\text{rec}}$.
2. **Turing-Recognizability:**
   - If $L_2 \in \mathcal{L}_{\text{RE}}$, then $L_1 \in \mathcal{L}_{\text{RE}}$.
   - If $L_1 \notin \mathcal{L}_{\text{RE}}$, then $L_2 \notin \mathcal{L}_{\text{RE}}$.
3. **Complements:**

$$L_1 \leq_{EE} L_2 \iff L_1^c \leq_{EE} L_2^c \quad \text{(using the exact same function } f\text{)}$$

Remark 4.8. Difference between $\leq_R$ and $\leq_{EE}$ on Recognizability: While $L_1 \leq_R L_2$ preserves decidability, it does **not** necessarily preserve Turing-recognizability (for instance, $L_{\text{diag}} \leq_R L_{\text{diag}}^c$ holds by flipping the result, but $L_{\text{diag}}^c \in \mathcal{L}_{\text{RE}}$ while $L_{\text{diag}} \notin \mathcal{L}_{\text{RE}}$). In contrast, an EE-reduction **preserves both** decidability and Turing-recognizability.

4.4 Canonical Undecidable Problems and Classic Reductions

Definition 4.9. The Halting Problem (H and H_ε):

- **General Halting Problem:**

$$H = \{\langle M \rangle \# w \mid M \text{ halts on input } w\}$$

- **Empty-Tape Halting Problem (Special Halting Problem):**

$$H_\varepsilon = \{\langle M \rangle \mid M \text{ halts on the empty input } \varepsilon\}$$

Example 4.10. Reduction $L_{\text{univ}} \leq_{EE} H$: Given an instance $\langle M \rangle \# w$ of L_{univ}, construct a Turing machine M' that simulates M on w. If M accepts w, M' halts. If M rejects w, M' enters an intentional infinite loop. Then:

$$\langle M \rangle \# w \in L_{\text{univ}} \iff M \text{ accepts } w \iff M' \text{ halts} \iff \langle M' \rangle \# w \in H$$

Since L_{univ} is undecidable, it follows that H is **undecidable**.

Example 4.11. Reduction $H \leq_{EE} H_\varepsilon$: Given an instance $\langle M \rangle \# w$ of H, define a reduction function $f(\langle M \rangle \# w) = \langle M_w \rangle$, where M_w is a newly constructed Turing machine that:

1. Ignores its own input tape.
2. Writes the fixed word w onto its tape.
3. Simulates M on w.

When started on the empty word ε, M_w writes w and simulates $M(w)$. Thus:

$$M_w \text{ halts on } \varepsilon \iff M \text{ halts on } w$$

Therefore, $\langle M \rangle \# w \in H \iff \langle M_w \rangle \in H_\varepsilon$. Because H is undecidable, H_ε is also **undecidable**.

4.5 The Emptiness Problem and Non-Recognizability

Definition 4.12. The Emptiness Problem ($L_\emptyset$): The language of all Turing machines that accept no words:

$$L_\emptyset = \{\langle M \rangle \mid L(M) = \emptyset\}$$

Its complement is $L_{\neq\emptyset} = \{\langle M \rangle \mid L(M) \neq \emptyset\}$.

Theorem 4.13. Status of $L_\emptyset$ and $L_{\neq\emptyset}$:

- $L_{\neq\emptyset}$ is **Turing-recognizable** ($L_{\neq\emptyset} \in \mathcal{L}_{\text{RE}}$), via dovetailing simulation across all possible words.
- $L_{\neq\emptyset}$ is **undecidable** ($H_\varepsilon \leq_{EE} L_{\neq\emptyset}$).

- $L_\emptyset$ is **not Turing-recognizable** ($L_\emptyset \notin \mathcal{L}_{\text{RE}}$).

4.6 Rice's Theorem

Definition 4.14. Semantic Property of Turing Machines: A property of Turing machines is called a **semantic property** (or language property) if it depends solely on the recognized language $L(M)$, not on the specific encoding or implementation details of M:

$$L(M_1) = L(M_2) \implies (\langle M_1 \rangle \in L_{\mathcal{P}} \iff \langle M_2 \rangle \in L_{\mathcal{P}})$$

A property $\mathcal{P} \subseteq \mathcal{L}_{\text{RE}}$ is called **non-trivial** if there is at least one recognizable language that has property $\mathcal{P}$ and at least one that does not:

$$\mathcal{P} \neq \emptyset \quad \text{and} \quad \mathcal{P} \neq \mathcal{L}_{\text{RE}}$$

Theorem 4.15. Rice's Theorem (Satz von Rice): Every non-trivial semantic property of Turing-recognizable languages is **undecidable**. That is, if $\emptyset \subsetneq \mathcal{P} \subsetneq \mathcal{L}_{\text{RE}}$, then:

$$L_{\mathcal{P}} = \{\langle M \rangle \mid L(M) \in \mathcal{P}\} \notin \mathcal{L}_{\text{rec}}$$

Remark 4.16. Power of Rice's Theorem: Rice's theorem immediately proves the undecidability of virtually every semantic question about program behavior without needing separate manual reductions:

- Is $L(M) = \emptyset$? (Undecidable)
- Is $L(M)$ finite? (Undecidable)
- Is $L(M)$ regular? (Undecidable)
- Does M accept the word 01? (Undecidable)

Warning (Syntactic properties are NOT covered): Rice's theorem only applies to semantic properties (languages). Syntactic properties (e.g., ,,Does M have more than 5 states¿‘ or ,,Does M ever execute transition $\delta(q_1, 0)$ on its transition diagram¿‘) are structural features of the code and are typically easily decidable.